

Q1 — Inventory Database

Products With Low Stock

Database Description

The inventory database is designed to manage products, stock levels, suppliers, and inventory movements. It helps organizations monitor available stock and identify products that need to be restocked.

Schema Details

Database: inventory

Table: Products

Column Name	Data Type	Constraints	Description
product_id	INT	PRIMARY KEY	Unique product ID
product_name	VARCHAR(100)	NOT NULL	Name of the product
category	VARCHAR(50)	NOT NULL	Product category
stock_quantity	INT	NOT NULL	Current quantity available
price	DECIMAL(10,2)	NOT NULL	Product price

Sample Data

product_id	product_name	category	stock_quantity	price
101	Wireless Mouse	Electronics	45	799.00
102	Keyboard	Electronics	8	1299.00
103	Office Chair	Furniture	25	6500.00
104	USB Cable	Electronics	5	299.00
105	Notebook	Stationery	60	120.00

Problem Statement

The inventory team wants to identify products that are running low on stock. Your task is to retrieve all products whose current stock quantity is **less than 10**.

The result should include the product_id, product_name, and stock_quantity, sorted by stock_quantity in ascending order.

Query to be Implemented

Write a query to return all products whose stock quantity is less than 10.

Output:

- product_id
- product_name
- stock_quantity

Sort the result by stock_quantity in ascending order.

✓ Expected Output

product_id	product_name	stock_quantity
104	USB Cable	5
102	Keyboard	8

Q2 — SmartMart Database

Products With High Prices

📁 Database Description

The smartmart database is designed to manage a retail supermarket's products, categories, customers, orders, and sales activities. It helps the organization analyze products, sales performance, and customer purchasing behavior.

📄 Schema Details

Database: smartmart

📊 Table: Products

Column Name	Data Type	Constraints	Description
product_id	INT	PRIMARY KEY	Unique product ID
product_name	VARCHAR(100)	NOT NULL	Name of the product
category	VARCHAR(50)	NOT NULL	Product category
price	DECIMAL(10,2)	NOT NULL	Product selling price
stock_quantity	INT	NOT NULL	Available stock

Sample Data

product_id	product_name	category	price	stock_quantity
201	Rice 5kg	Grocery	450.00	35
202	Coffee Maker	Appliances	3200.00	12
203	Washing Powder	Household	650.00	40
204	Mixer Grinder	Appliances	4500.00	8
205	Cooking Oil	Grocery	180.00	50

📌 Problem Statement

The SmartMart management team wants to identify expensive products for a special premium-product analysis.

Your task is to retrieve all products whose price is **greater than ₹2,000**.

The result should include the product_id, product_name, and price, sorted by price in **descending order**.

📌 Query to be Implemented

Write a query to return all products with a price greater than ₹2,000.

Output:

- product_id
- product_name
- price

Sort the result by price in descending order.

✓ Expected Output

product_id	product_name	price
204	Mixer Grinder	4500.00
202	Coffee Maker	3200.00

Q3 — Student Database

Students With High Marks

📁 Database Description

The student database is designed to manage student information, courses, examination results, and academic performance. It enables educational institutions to analyze student performance and identify high-performing students.

📄 Schema Details

Database: student

📊 Table: Students

Column Name	Data Type	Constraints	Description
student_id	INT	PRIMARY KEY	Unique student ID
student_name	VARCHAR(100)	NOT NULL	Name of the student
course	VARCHAR(50)	NOT NULL	Student's course
marks	INT	NOT NULL	Marks obtained
grade	VARCHAR(5)	—	Grade obtained

Sample Data

student_id	student_name	course	marks	grade
301	Aisha	SQL	78	B
302	Rahul	SQL	92	A
303	Neha	Python	85	A
304	Arjun	SQL	67	C
305	Priya	Python	95	A

📌 Problem Statement

The academic team wants to identify students who have demonstrated excellent academic performance.

Your task is to retrieve all students who scored **more than 80 marks**.

The result should include the student_id, student_name, and marks, sorted by marks in **descending order**.

Query to be Implemented

Write a query to return all students who scored more than 80 marks.

Output:

- student_id
- student_name
- marks

Sort the result by marks in descending order.

Expected Output

student_id	student_name	marks
305	Priya	95
302	Rahul	92
303	Neha	85

Recommended progression for you.

Since you know SQL **theory and straightforward SELECT queries**, I would make the questions gradually more challenging:

Level	Inventory	SmartMart	Student
Level 1	Low stock products	Expensive products	Students above 80
Level 2	Products in a particular category	Products within a price range	Students from a particular course
Level 3	Sort by stock/value	Sort by price	Sort by marks
Level 4	AND / OR conditions	Multiple conditions	Multiple conditions
Level 5	Aggregation	GROUP BY	Average marks
Level 6	HAVING	Category-wise sales	Course-wise performance
Level 7	Joins	Customer → Orders → Products	Students → Courses → Results
Level 8	Subqueries	Top-selling products	Students above average
Level 9	CTEs	Complex sales analysis	Ranking students
Level 10	Window functions	Business analytics	Student ranking

